

C++



CÓMO PROGRAMAR

NOVENA EDICIÓN

11

PAUL DEITEL
HARVEY DEITEL

8

Apuntadores

*Recibimos direcciones para
ocultar nuestro paradero.*

—Saki (H. H. Munro)

*Averigua la dirección mediante
la indirección.*

—William Shakespeare

*Muchas cosas, teniendo
referencia completa
A un consentimiento, pueden
trabajar en forma contraria*

—William Shakespeare

*¡Descubrirá que es una muy
buena práctica verificar siempre
sus referencias, señor!*

—Dr. Routh

Objetivos

En este capítulo aprenderá a:

- Distinguir qué son los apuntadores.
- Conocer las similitudes y diferencias entre los apuntadores y las referencias.
- Utilizar apuntadores para pasar argumentos a las funciones por referencia.
- Conocer las estrechas relaciones entre los apuntadores y los arreglos integrados.
- Utilizar cadenas basadas en apuntador.
- Utilizar los arreglos integrados.
- Utilizar las herramientas de C++11, incluyendo `nullptr` y las funciones `begin` y `end` de la Biblioteca estándar.



8.1	Introducción	8.6.2	Apuntador no constante a datos constantes
8.2	Declaraciones e inicialización de variables apunadores	8.6.3	Apuntador constante a datos no constantes
8.3	Operadores de apunadores	8.6.4	Apuntador constante a datos constantes
8.4	Paso por referencia mediante apunadores	8.7	Operador sizeof
8.5	Arreglos integrados	8.8	Expresiones y aritmética de apunadores
8.6	Uso de const con apunadores	8.9	Relación entre apunadores y arreglos integrados
8.6.1	Apuntador no constante a datos no constantes	8.10	Cadenas basadas en apuntador
		8.11	Conclusión

Resumen | Ejercicios de autoevaluación | Respuestas a los ejercicios de autoevaluación | Ejercicios
| Sección especial: construya su propia computadora

8.1 Introducción

En este capítulo hablaremos sobre los *apunadores*: una de las características más poderosas y desafiantes de usar del lenguaje de programación C++. Nuestros objetivos aquí son ayudarle a determinar cuándo es apropiado usar apunadores y mostrarle cómo usarlos de manera *correcta y responsable*.

En el capítulo 6, vimos que se pueden utilizar referencias para realizar el paso por referencia. Los apunadores también permiten el paso por referencia, y se pueden utilizar para crear y manipular estructuras dinámicas de datos que pueden crecer y reducirse, como las listas enlazadas, colas, pilas y árboles. En este capítulo explicaremos los conceptos básicos sobre los apunadores. En el capítulo 19 (en inglés en el sitio web) presentaremos ejemplos de cómo crear y usar estructuras de datos dinámicas que se implementan con apunadores.

También le mostraremos la estrecha relación entre los *arreglos integrados* y los apunadores. C++ heredó los arreglos integrados del lenguaje de programación C. Como vimos en el capítulo 7, las clases `array` y `vector` de la Biblioteca estándar de C++ proporcionan implementaciones de los arreglos como objetos completos; de hecho, tanto `array` como `vector` almacenan sus elementos en arreglos integrados. *En los proyectos nuevos de desarrollo de software, es mejor usar objetos `array` y `vector` en vez de los arreglos integrados.*

De manera similar, C++ en realidad ofrece dos tipos de cadenas: objetos de la clase `string` (que hemos estado usando desde el capítulo 3) y *cadenas basadas en apuntador, estilo C* (cadenas de C). Este capítulo introduce brevemente las cadenas C para que el lector amplíe su conocimiento sobre los apunadores y los arreglos integrados. Las cadenas de C se utilizaban ampliamente en software antiguo de C y C++. Hablaremos con detalle sobre ellas en el apéndice F. *En los proyectos nuevos de desarrollo de software, es mejor usar objetos de la clase `string`.*

En el capítulo 12 examinaremos el uso de los apunadores con los objetos de clases, en donde veremos que lo que se denomina “procesamiento polimórfico” de la programación orientada a objetos se lleva a cabo mediante apunadores y referencias.

8.2 Declaraciones e inicialización de variables apunadores

Indirección

Las variables apunadores contienen *direcciones de memoria* como sus valores. Por lo general, una variable contiene *directamente* un valor específico. Un apuntador contiene la *dirección de memoria* de una variable que, a su vez, contiene un valor específico. En este sentido, el nombre de una variable **hace referencia directa a un valor**, y un apuntador **hace referencia indirecta a un valor** (figura 8.1). Al

proceso de hacer referencia a un valor a través de un apuntador se le conoce comúnmente como **indirección**. Por lo general, los diagramas representan un apuntador en forma de una *flecha* que parte de la *variable que contiene una dirección*, hasta la *variable ubicada en esa dirección* de memoria.

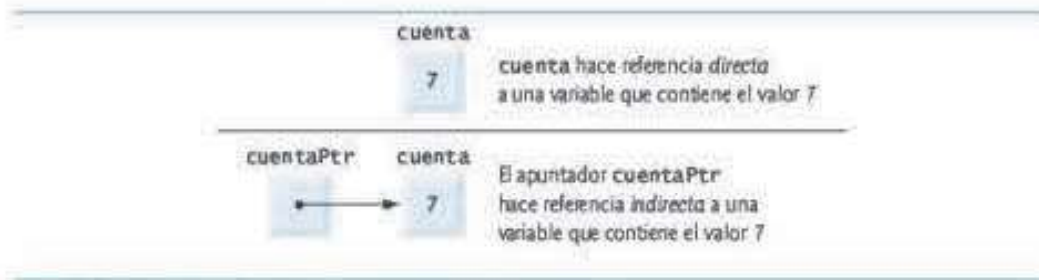


Fig. 8.1 Referencia directa e indirecta a una variable.

Declaración de apuntadores

Al igual que las demás variables, los apuntadores se deben declarar *antes* de poder usarlos. Por ejemplo, para el apuntador en la figura 8.1, la declaración

```
int *cuentaPtr, cuenta;
```

declara la variable `cuentaPtr` como de tipo `int *` (es decir, un apuntador a un valor `int`) y se lee así (de derecha a izquierda): “`cuentaPtr` es un apuntador a un `int`”. Además, la variable `cuenta` en la declaración anterior se declara como un `int`, y *no* como un apuntador a un `int`. El `*` en la declaración se aplica *sólo* a `cuentaPtr`. A cada variable que se declara como apuntador se le *debe* anteponer un asterisco (`*`). Por ejemplo, la declaración

```
double *xPtr, *yPtr;
```

indica que tanto `xPtr` como `yPtr` son apuntadores a valores `double`. Cuando el `*` aparece en una declaración, *no* es un operador; en vez de ello, indica que la variable que se está declarando es un apuntador. Los apuntadores se pueden declarar de manera que apunten a objetos de *cualquier* tipo de datos.

Error común de programación 8.1



Asumir que el `*` que se utiliza para declarar a un apuntador se distribuye a todos los nombres de variables en la lista separada por comas de variables de una declaración puede provocar errores. Cada apuntador se debe declarar con el `*` antepuesto al nombre (ya sea con o sin un espacio entre ellos). Si declaramos sólo una variable por cada declaración, evitamos estos tipos de errores y mejoramos la legibilidad de los programas.

Buena práctica de programación 8.1



Aunque no es un requerimiento, incluir las letras `Ptr` en los nombres de las variables apuntadores deja claro que estas variables son apuntadores, y que deben tratarse de manera acorde.

Inicialización de apuntadores



Los apuntadores se deben inicializar con `nullptr` (nuevo en C++11) o con una dirección del tipo correspondiente, ya sea cuando se declaran o en una asignación. Un apuntador con el valor `nullptr` “no apunta a nada”, y se conoce como **apuntador nulo**. De aquí en adelante, cuando hagamos referencia a un “apuntador nulo” nos estaremos refiriendo a un apuntador con el valor `nullptr`.



Tip para prevenir errores 8.1

Inicialice todos los apuntadores para evitar que apunten hacia áreas desconocidas o no inicializadas de la memoria.

Apuntadores nulos anteriores a C++11

En versiones anteriores de C++, el valor especificado para un apuntador nulo era 0 o NULL. Este último se define en varios encabezados de la biblioteca estándar para representar el valor 0. Inicializar un apuntador a NULL es equivalente a inicializar un apuntador a 0, pero antes de C++11, se utilizaba 0 por convención. El 0 es el *único* valor entero que puede asignarse directamente a una variable apuntador sin primero *convertir* el entero a un tipo apuntador.

8.3 Operadores de apuntadores

Operador dirección (&)

El **operador dirección (&)** es un operador unario que *obtiene la dirección de memoria de su operando*. Por ejemplo, teniendo en cuenta las siguientes declaraciones:

```
int y = 5; // declara la variable y
int *yPtr = nullptr; // declara la variable apuntador yPtr
```

la instrucción

```
yPtr = &y; // asigna la dirección de y a yPtr
```

asigna la dirección de la variable y a la variable apuntador yPtr. Entonces, se dice que la variable yPtr “apunta a” y. Ahora, yPtr hace referencia *indirecta* al valor de la variable y. Observe que el uso del signo & en la instrucción anterior *no* es el mismo que el uso del & en la *declaración de una variable de referencia*, a la cual *siempre* se le antepone el nombre de un tipo de datos. Al declarar una referencia, el & forma parte del tipo. En una expresión como &y, el & es el *operador dirección*.

La figura 8.2 muestra una representación de la memoria después de la asignación anterior. La “relación de señalamiento” se indica mediante el dibujo de una flecha desde el cuadro que representa al apuntador yPtr en la memoria, hasta el cuadro que representa a la variable y en la memoria.

La figura 8.3 muestra otra representación del apuntador en memoria, con la variable entera y almacenada en la ubicación de memoria 600000 y la variable apuntador yPtr almacenada en la ubicación de memoria 500000. El operando del operador dirección debe ser un *lvalue*; el operador dirección *no se puede* aplicar a constantes o expresiones que produzcan valores temporales (como los resultados de los cálculos).



Fig. 8.2 Representación gráfica de un apuntador que apunta a una variable en memoria.



Fig. 8.3 Representación de y y yPtr en memoria.

Operador de indirección (*)

El **operador *** unario, que se conoce comúnmente como el **operador de indirección** u **operador de desreferencia**, *devuelve un lvalue que representa al objeto al que apunta su operando apuntador*. Por ejemplo (haciendo referencia otra vez a la figura 8.2), la instrucción

```
cout << *yPtr << endl;
```

imprime el valor de la variable *y* (en este caso, 5), al igual que la instrucción.

```
cout << y << endl;
```

Al proceso de utilizar el *** de esta manera, se le conoce como **desreferenciar un apuntador**. Un *apuntador desreferenciado* también se puede usar en el lado *izquierdo* de una instrucción de asignación, como en

```
*yPtr = 9;
```

lo cual asignaría 9 a *y* en la figura 8.3. El *apuntador desreferenciado* también se puede utilizar para recibir un valor de entrada, como en

```
cin >> *yPtr;
```

lo cual coloca el valor de entrada en *y*.



Error común de programación 8.2

Al desreferenciar un apuntador que no se haya inicializado se produce un comportamiento indefinido, el cual podría producir un error fatal en tiempo de ejecución. Esto también podría conducir a la modificación accidental de datos importantes y permitir que el programa se ejecutara por completo, lo que posiblemente produzca resultados incorrectos.



Tip para prevenir errores 8.2

Desreferenciar un apuntador nulo produce un comportamiento indefinido y, por lo general, es un error en tiempo fatal de ejecución, por lo que debe asegurarse de que un apuntador no sea nulo antes de desreferenciarlo.

Uso de los operadores dirección (&) e indirección (*)

El programa de la figura 8.4 demuestra los operadores *&* y *** para apuntadores. Las ubicaciones de memoria se imprimen mediante *<<* en este ejemplo como enteros *hexadecimales* (base 16) (en el apéndice D, Sistemas numéricos, podrá obtener más información acerca de los enteros hexadecimales). Las direcciones de memoria que imprime este programa son *dependientes de la plataforma*, por lo que tal vez usted obtenga distintos resultados cuando ejecute el programa. La dirección de *a* (línea 11) y el valor de *aPtr* (línea 12) son idénticos en la salida, lo cual confirma que la dirección de *a* se asigna en definitiva a la variable apuntador *aPtr*.

```
1 // Fig. 8.4: fig08_04.cpp
2 // Los operadores & y * de los apuntadores.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     int a = 7; // se asigna un 7 a la variable a
9     int *aPtr = &a; // aPtr se inicializa con la dirección de la variable int a
10
11     cout << "La dirección de a es " << &a
12         << "\nEl valor de aPtr es " << aPtr;
13     cout << "\nEl valor de a es " << a
14         << "\nEl valor de *aPtr es " << *aPtr << endl;
15 }
```

Fig. 8.4 Los operadores *&* y *** de los apuntadores (parte 1 de 2).

```

La dirección de a es 002DFD80
El valor de aPtr es 002DFD80

El valor de a es 7
El valor de *aPtr es 77

```

Fig. 8.4 | Los operadores & y * de los apuntadores (parte 2 de 2).

Precedencia y asociatividad de los operadores presentados hasta ahora

La figura 8.5 lista la precedencia y asociatividad de los operadores presentados hasta ahora. El operador dirección (&) y el operador desreferencia (*) son *operadores unarios* en el cuarto nivel.

Operadores	Asociatividad	Tipo
:: O	izquierda a derecha <i>[Vea la precaución en la figura 2.10 con respecto al agrupamiento de paréntesis].</i>	primario
O [] ++ -- static_cast<type>(operand)	izquierda a derecha	postfijo
++ -- + - ! & *	derecha a izquierda	unario (prefijo)
* / %	izquierda a derecha	multiplicativa
+ -	izquierda a derecha	aditiva
<< >>	izquierda a derecha	inserción/extracción
< <= > >=	izquierda a derecha	relacional
== !=	izquierda a derecha	igualdad
&&	izquierda a derecha	AND lógico
	izquierda a derecha	OR lógico
?:	derecha a izquierda	condicional
= += -= *= /= %=	derecha a izquierda	asignación
,	izquierda a derecha	coma

Fig. 8.5 | Precedencia y asociatividad de los operadores vistos hasta ahora.

8.4 Paso por referencia mediante apuntadores

En C++ hay tres formas de pasar argumentos a una función: paso por valor, paso por referencia con argumentos tipo referencia y **paso por referencia con argumentos tipo apuntador**. En el capítulo 6 comparamos y contrastamos el paso por referencia con argumentos tipo referencia y el paso por valor. En esta sección, explicaremos el paso por referencia con argumentos tipo apuntador.

Como vimos en el capítulo 6, se puede utilizar `return` para devolver *un valor* de una función a la que se llamó, o simplemente devolver el *control*. También vimos que se pueden pasar argumentos a una función mediante el uso de parámetros de referencia, los cuales permiten a la función llamada *modificar los valores originales de los argumentos en la función que hizo la llamada*. Los parámetros de referencia también permiten a los programas pasar *objetos de datos grandes* a una función, y evitar la sobrecarga de pasar los objetos por valor (que, desde luego, copia el objeto). Al igual que las referencias, los apuntadores también se pueden usar para modificar una o más variables en la función que hace la llamada, o pasar apuntadores a objetos de datos grandes para evitar la sobrecarga de pasar los objetos por valor.

Podemos usar apuntadores y el operador indirección (*) para realizar el paso por referencia (en forma idéntica al paso por referencia en los programas en C, ya que éste no tiene referencias). Cuando se llama a una función con un argumento que se debe modificar, se pasa la *dirección* del argumento. Por lo general, para realizar esto se aplica el operador dirección (&) al nombre de la variable cuyo valor se va a modificar.

Un ejemplo de paso por valor

Las figuras 8.6 y 8.7 presentan dos versiones de una función que eleva un entero al cubo. La figura 8.6 pasa la variable *numero* por valor (línea 14) a la función *cuboPorValor* (líneas 19 a 22), que eleva su argumento al cubo y pasa el nuevo valor de vuelta a *main*, usando una instrucción *return* (línea 21). El nuevo valor se asigna a *numero* (línea 14) en *main*. La función que llama tiene la oportunidad de examinar el resultado de la llamada a la función *antes* de modificar el valor de la variable *numero*. Por ejemplo, podríamos haber almacenado el resultado de *cuboPorValor* en otra variable, para después examinar su valor y asignar el resultado a *numero*, sólo después de determinar que el valor devuelto era razonable.

```

1 // Fig. 8.6: fig08_06.cpp
2 // Uso del paso por valor para elevar al cubo el valor de una variable.
3 #include <iostream>
4 using namespace std;
5
6 int cuboPorValor( int ); // prototipo
7
8 int main()
9 {
10     int numero = 5;
11
12     cout << "El valor original de numero es " << numero;
13
14     numero = cuboPorValor( numero ); // pasa el numero por valor a cuboPorValor
15     cout << "\nEl nuevo valor de numero es " << numero << endl;
16 } // fin de main
17
18 // calcula y devuelve el cubo del argumento entero
19 int cuboPorValor( int n )
20 {
21     return n * n * n; // eleva al cubo la variable local n y devuelve el resultado
22 } // fin de la función cuboPorValor

```

```

El valor original de numero es 5
El nuevo valor de numero es 125

```

Fig. 8.6 | Uso del paso por valor para elevar al cubo el valor de una variable.

Un ejemplo de paso por referencia con apuntadores

En la figura 8.7 se pasa la variable *numero* a la función *cuboPorReferencia* mediante el uso del *paso por referencia con un argumento tipo apuntador* (línea 15); la *dirección* de *numero* se pasa a la función. La función *cuboPorReferencia* (líneas 21 a 24) especifica el parámetro *nPtr* (un apuntador a *int*) para recibir su argumento. La función *usa el apuntador desreferenciado* para elevar al cubo el valor al que apunta *nPtr* (línea 23). Esto modifica *directamente* el valor de *numero* en *main* (línea 11). La línea 23 es equivalente a

```
*nPtr = (*nPtr) * (*nPtr) * (*nPtr); // eleva *nPtr al cubo
```

```

1 // Fig. 8.7: fig08_07.cpp
2 // Uso del paso por referencia con un argumento apuntador para elevar
3 // al cubo el valor de una variable.
4 #include <iostream>
5 using namespace std;
6
7 void cuboPorReferencia( int * ); // prototipo
8
9 int main()
10 {
11     int numero = 5;
12
13     cout << "El valor original de numero es " << numero;
14
15     cuboPorReferencia( &numero ); // pasa la dirección de numero
16                                     // a cuboPorReferencia
17
18     cout << "\nEl nuevo valor de numero es " << numero << endl;
19 } // fin de main
20
21 // calcula el cubo de *nPtr; modifica la variable numero en main
22 void cuboPorReferencia( int *nPtr )
23 {
24     *nPtr = *nPtr * *nPtr * *nPtr; // eleva *nPtr al cubo
25 } // fin de la función cuboPorReferencia

```

```

El valor original de numero es 5
El nuevo valor de numero es 125

```

Fig. 8.7 | Uso del paso por referencia con un argumento tipo apuntador para elevar al cubo el valor de una variable.

Una función que recibe una *dirección* como argumento debe definir un *parámetro tipo apuntador* para recibir la dirección. Por ejemplo, el encabezado para la función `cuboPorReferencia` (línea 21) especifica que `cuboPorReferencia` debe recibir la dirección de una variable `int` (es decir, un apuntador a un `int`) como argumento, debe almacenar la dirección en forma local en `nPtr` y *no* debe devolver un valor.

El prototipo de función para `cuboPorReferencia` (línea 7) contiene `int *` entre paréntesis. Al igual que con otros tipos de variables, no es necesario incluir los *nombres* de los parámetros tipo apuntador en los prototipos. El compilador *ignora* los nombres de los parámetros incluidos para fines de documentación.

Perspectiva: todos los argumentos se pasan por valor

En C++, *todos* los argumentos *siempre* se pasan por valor. Al pasar una variable por referencia con un apuntador *en realidad no se pasa algo por referencia: se pasa un apuntador a esa variable por valor y se copia* al parámetro del apuntador correspondiente de esa función. Así, la función a la que se llamó puede acceder a esa variable en la función que hizo la llamada, con sólo desreferenciar el apuntador, logrando así el *paso por referencia*.

Análisis gráfico del paso por valor y del paso por referencia

Las figuras 8.8 y 8.9 analizan gráficamente la ejecución de los programas de las figuras 8.6 y 8.7, respectivamente. En los diagramas, los valores en los rectángulos sobre una expresión o variable dada representan el valor de esa expresión o variable. La columna derecha de cada diagrama muestra las funciones `cuboPorValor` (figura 8.6) y `cuboPorReferencia` (figura 8.7) *sólo* cuando se están ejecutando.

Paso 1: antes de que main llame a cuboPorValor:

```
int main()
{
    int numero = 5;
    numero = cuboPorValor( numero );
}
```

numero
5

Paso 2: después de que cuboPorValor recibe la llamada:

```
int main()
{
    int numero = 5;
    numero = cuboPorValor( numero );
}
```

numero
5

```
int cuboPorValor( int n )
{
    return n * n * n;
}
```

n
5

Paso 3: después de que cuboPorValor eleva al cubo el parámetro n y antes de que cuboPorValor regrese a main:

```
int main()
{
    int numero = 5;
    numero = cuboPorValor( numero );
}
```

numero
5

```
int cuboPorValor( int n )
{
    125
    return n * n * n;
}
```

n
5

Paso 4: después de que cuboPorValor regresa a main y antes de asignar el resultado a numero:

```
int main()
{
    int numero = 5;
    numero = cuboPorValor( numero );
}
```

numero
5

125

Paso 5: después de que main completa la asignación para numero:

```
int main()
{
    int numero = 5;
    125
    numero = cuboPorValor( numero );
}
```

numero
125

Fig. 8.8 | Análisis del paso por valor del programa de la figura 8.6.

Paso 1: antes de que main llame a cuboPorReferencia:

```
int main()
{
    int numero = 5;
    cuboPorReferencia( &numero );
}
```

numero
5

Paso 2: después de que cuboPorReferencia recibe la llamada y antes de que *nPtr se eleve al cubo:

```
int main()
{
    int numero = 5;
    cuboPorReferencia( &numero );
}
```

numero
5

```
void cuboPorReferencia( int *nPtr )
{
    *nPtr = *nPtr * *nPtr * *nPtr;
}
```

nPtr
la llamada establece este apuntador

Paso 3: antes de que se asigne a *nPtr el resultado del cálculo $5 * 5 * 5$:

```
int main()
{
    int numero = 5;
    cuboPorReferencia( &numero );
}
```

numero
5

```
void cuboPorReferencia( int *nPtr )
{
    125
    *nPtr = *nPtr * *nPtr * *nPtr;
}
```

nPtr

Paso 4: después de asignar 125 a *nPtr y antes de que el control del programa regrese a main:

```
int main()
{
    int numero = 5;
    cuboPorReferencia( &numero );
}
```

numero
125

```
void cuboPorReferencia( int *nPtr )
{
    125
    *nPtr = *nPtr * *nPtr * *nPtr;
}
```

nPtr
la función llamada modifica la variable de la función que hizo la llamada

Paso 5: después de que cuboPorReferencia regresa a main:

```
int main()
{
    int numero = 5;
    cuboPorReferencia( &numero );
}
```

numero
125

Fig. 8.9 | Análisis del paso por referencia (con un argumento tipo apuntador) del programa de la figura 8.7.

8.5 Arreglos integrados

En el capítulo 7, usamos la plantilla de clase `array` para representar listas y tablas de valores de *tamaño fijo*. También usamos la plantilla de clase `vector`, que es similar a `array`, sólo que también puede crecer (o encogerse como veremos en el capítulo 15) en forma dinámica para dar cabida a más o menos elementos. En esta sección presentamos los *arreglos integrados*, que también son estructuras de datos de *tamaño fijo*.

Declaración de un arreglo integrado

Para especificar el tipo y el número de elementos requeridos por un arreglo integrado, use una declaración de la forma:

```
tipo nombreArreglo[ tamañoArreglo ];
```

El compilador reserva la cantidad de memoria apropiada. El *tamañoArreglo* debe ser una constante entera mayor que cero. Por ejemplo, para indicar al compilador que reserve 12 elementos para el arreglo integrado de valores `int` llamado `c`, use la declaración:

```
int c[ 12 ]; // c es un arreglo integrado de 12 enteros
```

Acceso a los elementos de un arreglo integrado

Al igual que con los objetos `array`, se utiliza el operador subíndice (`[]`) para acceder a los elementos individuales de un arreglo integrado. En el capítulo 7 vimos que el operador subíndice (`[]`) *no* cuenta con comprobación de límites para los objetos `array`; esto también aplica para los arreglos integrados.

Inicialización de arreglos integrados

Podemos inicializar los elementos de un arreglo integrado mediante el uso de una *lista inicializadora*. Por ejemplo,

```
int n[ 5 ] = { 10, 20, 30, 10, 40 };
```

crea un arreglo integrado de cinco valores `int` y los inicializa con los valores en la lista inicializadora. Si proporciona menos inicializadores que el número de elementos, los elementos restantes se *inicializan por valor*; los tipos numéricos fundamentales se establecen con 0, los valores `bool` se establecen con `false`, los apuntadores se establecen con `nullptr` y los objetos de clases se inicializan mediante sus constructores predeterminados. Si proporciona demasiados inicializadores, se produce un error de compilación. La nueva sintaxis de inicialización de listas de C++11 que presentamos en el capítulo 4 se basa en la sintaxis de listas inicializadoras de los arreglos integrados.

Si se *omite* el tamaño de un arreglo integrado de una declaración con una lista inicializadora, el compilador ajusta el tamaño del arreglo integrado con el número de elementos en la lista inicializadora. Por ejemplo,

```
int n[] = { 50, 20, 30, 10, 40 };
```

crea un arreglo de cinco elementos.



Tip para prevenir errores 8.3



Especifique siempre el tamaño de un arreglo integrado, incluso cuando se proporcione una lista inicializadora. Esto permite al compilador asegurar que no se proporcionen demasiados inicializadores.

Paso de arreglos integrados a funciones

El valor del nombre de un arreglo integrado se convierte de manera implícita a la dirección del primer elemento del arreglo integrado. Así, `nombreArreglo` se convierte implícitamente en `&nombreArreglo[0]`.

Por esta razón, no es necesario tomar la dirección (&) de un arreglo integrado para pasarla a una función; sólo hay que pasar el nombre del arreglo integrado. Como vimos en la sección 8.4, una función que recibe un apuntador a una variable en la función que hizo la llamada puede *modificar* esa variable en la función que hizo la llamada. Para los arreglos integrados, esto significa que la función que hizo la llamada puede modificar *todos* los elementos de un arreglo integrado en la función que hizo la llamada; a menos que la función anteponga la palabra `const` al parámetro del arreglo integrado correspondiente, para indicar que los elementos *no* deben modificarse.



Observación de Ingeniería de Software 8.1

Aplicar el calificador de tipo `const` a un parámetro de arreglo integrado en una definición de función para evitar que el arreglo integrado original se modifique en el cuerpo de la función es otro ejemplo del principio de menor privilegio. Las funciones no deben tener la capacidad de modificar un arreglo integrado, a menos que sea absolutamente necesario.

Declaración de parámetros de arreglos integrados

Podemos declarar un parámetro de arreglo integrado en el encabezado de una función, como se muestra a continuación:

```
int sumaElementos( const int valores[], const size_t numeroDeElementos )
```

lo cual indica que el primer argumento de la función debe ser un arreglo integrado unidimensional de valores `int` que la función *no* debe modificar. A diferencia de los objetos `array`, los arreglos integrados no conocen su propio tamaño, por lo que una función que procesa un arreglo integrado debería tener parámetros para recibir el arreglo integrado y su tamaño.

El encabezado anterior también puede escribirse así:

```
int sumaElementos( const int *valores, const size_t numeroDeElementos )
```

El compilador no distingue entre una función que recibe un apuntador y una que recibe un arreglo integrado. Desde luego, esto significa que la función debe “saber” cuando está recibiendo un arreglo integrado o simplemente una variable individual que se está pasando por referencia. Cuando el compilador encuentra el parámetro de una función para un arreglo integrado unidimensional de la forma `const int valores[]`, convierte el parámetro a la notación de apuntador `const int *valores` (es decir, “valores es un apuntador a una constante entera”). Estas formas de declarar un parámetro de arreglo integrado unidimensional son intercambiables: por cuestión de *claridad*, es mejor usar la notación `[]` cuando la función espera un argumento de arreglo integrado.

C++11: Las funciones `begin` y `end` de la Biblioteca estándar

En la sección 7.7, le mostramos cómo ordenar un objeto `array` con la función `sort` de la Biblioteca estándar de C++. Ordenamos un arreglo de objetos `string` llamado `colores` de la siguiente manera:

```
sort( colores.begin(), colores.end() ); // ordena el contenido de colores
```

Las funciones `begin` y `end` de la clase `array` especifican que hay que ordenar todo el arreglo. La función `sort` (y muchas otras funciones de la Biblioteca estándar de C++) también puede aplicarse a los arreglos integrados. Por ejemplo, para ordenar el arreglo integrado `n` que se mostró anteriormente en esta sección, podemos escribir:

```
sort( begin( n ), end( n ) ); // ordena el contenido del arreglo integrado n
```

Las nuevas funciones `begin` y `end` de C++11 (del encabezado `<iterator>`) reciben cada una un arreglo integrado como argumento, y devuelven un apuntador que puede usarse para representar rangos de elementos para procesar en funciones de la Biblioteca estándar de C++, como `sort`.



Limitaciones de los arreglos integrados

Los arreglos integrados tienen varias limitaciones:

- *No pueden compararse* mediante el uso de los operadores relacionales y de igualdad; debemos usar un ciclo para comparar dos arreglos integrados, elemento por elemento.
- *No pueden asignarse* unos a otros.
- *No conocen su propio tamaño*; por lo general, una función que procesa un arreglo integrado recibe como argumentos *tanto el nombre del arreglo integrado como su tamaño*.
- *No cuentan con comprobación de límites automática*: debemos asegurarnos de que las expresiones que accedan a los arreglos usen subíndices que se encuentren dentro de los límites del arreglo integrado.

Los objetos de las plantillas de clases `array` y `vector` son más seguros, robustos y proporcionan más herramientas que los arreglos integrados.

Algunas veces se requieren arreglos integrados

En el código de C++ contemporáneo, es más conveniente usar los objetos `array` (o `vector`), que son más robustos, para representar listas y tablas de valores. Sin embargo, existen casos en los que se *deben* usar los arreglos integrados, como al procesar los **argumentos de línea de comandos** de un programa. Para suministrar argumentos de línea de comandos a un programa, hay que colocarlos después del nombre del programa cuando se vaya a ejecutar desde la línea de comandos. Por lo general, dichos argumentos pasan opciones a un programa. Por ejemplo, en una computadora Windows el comando

```
dir /p
```

usa el argumento `/p` para listar el contenido del directorio actual, haciendo pausa después de cada pantalla de información. De manera similar, en OS X o Linux, el siguiente comando usa el argumento `-la` para listar el contenido del directorio actual, con detalles acerca de cada archivo y directorio:

```
ls -la
```

Los argumentos de línea de comandos se pasan a `main` como un arreglo integrado de cadenas basadas en apuntador (sección 8.10). El apéndice F muestra cómo procesar argumentos de la línea de comandos.

8.6 Uso de `const` con apuntadores

Recuerde que el calificador `const` nos permite informar al compilador que el valor de una variable específica *no* se debe modificar. Existen muchas posibilidades para usar (o *no* usar) `const` con los parámetros de funciones, así que ¿cómo elegir la más apropiada de estas posibilidades? Hay que dejar que el *principio del menor privilegio* sea nuestro guía. Siempre debemos otorgar a una función el *suficiente* acceso a los datos en sus parámetros para que pueda realizar su tarea especificada, pero *nada más*. En esta sección veremos cómo combinar `const` con las declaraciones de apuntadores para hacer valer el principio del menor privilegio.

En el capítulo 6 explicamos que, cuando se llama a una función mediante el paso por valor, se pasa una *copia* del argumento a la función. Si la copia se *modifica* en la función a la que *se llamó*, el valor original se mantiene *sin cambios* en la función que hizo la llamada. En muchos casos, incluso la copia del valor del argumento *no* debe alterarse en la función a la que se llamó.

Considere una función que recibe un apuntador al elemento inicial de un arreglo integrado y su tamaño como argumentos, y por consiguiente imprime los elementos del arreglo integrado. Dicha función debería iterar a través de los elementos e imprimir cada uno por separado. El tamaño del arreglo integrado se utiliza en el cuerpo de la función para determinar el subíndice más alto del arreglo, de ma-

nera que el ciclo pueda terminar cuando se complete la impresión. El tamaño del arreglo no necesita cambiar en el cuerpo de la función, por lo que debe declararse como `const` para *asegurar* que no cambie. Desde luego, como el arreglo integrado sólo se va a imprimir, también debe declararse como `const`. En especial, esto es importante ya que los arreglos integrados *siempre* pasan por referencia, y podrían modificarse fácilmente en la función a la que se llama. Un intento de modificar un valor `const` es un error de compilación.



Observación de Ingeniería de Software 8.2

Si un valor no cambia (o no debe cambiar) en el cuerpo de una función que lo recibe, el parámetro se debe declarar `const`.



Tip para prevenir errores 8.4

Antes de usar una función, compruebe su prototipo para determinar los parámetros que puede y no puede modificar.

Hay cuatro maneras de pasar un apuntador a una función: un *apuntador no constante a datos no constantes*, un *apuntador no constante a datos constantes* (figura 8.10), un *apuntador constante a datos no constantes* (figura 8.11) y un *apuntador constante a datos constantes* (figura 8.12). Cada combinación proporciona un nivel distinto de privilegios de acceso.

8.6.1 Apuntador no constante a datos no constantes

El mayor nivel de acceso se otorga mediante un *apuntador no constante a datos no constantes*; los datos se pueden modificar a través del apuntador desreferenciado, y el apuntador se puede modificar para que apunte a otros datos. La declaración para dicho apuntador (por ejemplo, `int *cuentaPtr`) no incluye `const`.

8.6.2 Apuntador no constante a datos constantes

Un *apuntador no constante a datos constantes* es un apuntador que se puede modificar para apuntar a cualquier elemento de datos del tipo apropiado, pero los datos a los que apunta *no se pueden* modificar a través de ese apuntador. Dicho apuntador podría usarse para recibir un argumento tipo arreglo integrado para una función que pueda leer los elementos, pero *no* modificarlos. Cualquier intento por modificar los datos en la función resulta en un error de compilación. La declaración para dicho apuntador coloca `const` a la izquierda del tipo del apuntador, como en:

```
const int *cuentaPtr;
```

La declaración se lee de *derecha a izquierda* como “cuentaPtr es un apuntador a una constante entera” o, en forma más precisa, “cuentaPtr es un apuntador *no constante* a una constante entera”.

En la figura 8.10 se muestra el mensaje de error de compilación de GNU C++ que se produce al tratar de compilar una función que recibe un *apuntador no constante a datos constantes*, y después trata de usar ese apuntador para modificar los datos.

```
1 // Fig. 8.10: fig08_10.cpp
2 // Intento de modificar los datos a través de un
3 // apuntador no constante a datos constantes.
4
5 void f( const int * ); // prototipo
6
7 int main()
8 {
```

Fig. 8.10 | Intento de modificar datos a través de un apuntador no constante a datos const (parte 1 de 2).

```

9   int y = 0;
10
11   f(&y); // f intentará realizar una modificación ilegal
12 } // fin de main
13
14 // una variable constante no se puede modificar a través de xPtr
15 void f(const int *xPtr)
16 {
17     *xPtr = 100; // error: no se puede modificar un objeto const
18 } // fin de la función f

```

Mensaje de error del compilador GNU C++:

```

fig08_10.cpp: In function 'void f(const int*)':
fig08_10.cpp:17:12: error: assignment of read-only location '* xPtr'

```

Fig. 8.10 | Intento de modificar datos a través de un apuntador no constante a datos const (parte 2 de 2).

Cuando se llama a una función con un arreglo integrado como argumento, su contenido se pasa efectivamente por referencia debido a que el nombre del arreglo integrado puede convertirse de manera implícita en la dirección del primer elemento del arreglo integrado. Sin embargo y *de manera predeterminada*, los objetos como arreglos y vectores se pasan por valor; se pasa una copia del objeto completo. Para ello se requiere la sobrecarga en tiempo de ejecución de crear una copia de cada elemento de datos en el objeto y almacenarla en la pila de llamadas a funciones. Cuando se pasa un apuntador a un objeto, sólo debe realizarse una copia de la dirección del objeto; el objeto en sí no se copia.



Tip de rendimiento 8.1

Si no es necesario modificarlos por la función a la que se llamó, pase objetos grandes utilizando apuntadores a datos constantes o referencias a datos constantes, para obtener los beneficios de rendimiento del paso por referencia y evitar la sobrecarga de la copia del paso por valor.



Observación de Ingeniería de Software 8.3

Hay que pasar objetos grandes usando apuntadores a datos constantes, o referencias a datos constantes, para obtener la seguridad del paso por valor.



Observación de Ingeniería de Software 8.4

Use el paso por valor para pasar argumentos de tipos fundamentales (como int, double, etc.) a una función, a menos que la función que hace la llamada requiera de manera explícita que la función a la que se llamó pueda modificar directamente el valor en la función que hizo la llamada. Este es otro ejemplo del principio de menor privilegio.

8.6.3 Apuntador constante a datos no constantes

Un **apuntador constante a datos no constantes** es un apuntador que siempre apunta a la misma ubicación de memoria; los datos en esa ubicación se pueden modificar a través del apuntador. Los apuntadores que se declaran como const se deben inicializar a la hora de declararse, pero si el apuntador es un parámetro de función, se inicializa con el apuntador que se pasa a la función.

El programa de la figura 8.11 trata de modificar un apuntador constante. En la línea 11 se declara el apuntador ptr de tipo int * const. La declaración se lee de derecha a izquierda como "ptr es un

apuntador constante a un entero no constante". El apuntador se *inicializa* con la dirección de la variable entera `x`. En la línea 14 se hace un intento por *asignar* la dirección de `y` a `ptr`, pero el compilador genera un mensaje de error. No ocurre ningún error cuando en la línea 13 se asigna el valor 7 a `*ptr`; el valor no constante al que apunta `ptr` se *puede* modificar mediante el uso de `ptr` desreferenciado, aun cuando el mismo `ptr` se haya declarado como `const`.

```

1 // Fig. 8.11: fig08_11.cpp
2 // Intento de modificar un apuntador constante a datos no constantes.
3
4 int main()
5 {
6     int x, y;
7
8     // ptr es un apuntador constante a un entero que se puede
9     // modificar a través de ptr, pero ptr siempre apunta a la
10    // misma ubicación de memoria.
11    int * const ptr = &x; // el apuntador const se debe inicializar
12
13    *ptr = 7; // se permite: *ptr no es const
14    ptr = &y; // error: ptr es const; no se puede asignar a una nueva dirección
15 } // fin de main

```

Mensaje de error del compilador Microsoft Visual C++:

you cannot assign to a variable that is const

Fig. 8.11 | Intento de modificar un apuntador constante a datos no constantes.

8.6.4 Apuntador constante a datos constantes

El *mínimo* privilegio de acceso se otorga mediante un **apuntador constante a datos constantes**. Dicho apuntador *siempre* apunta a la *misma* ubicación de memoria, y los datos en esa ubicación de memoria *no se pueden* modificar mediante el uso del apuntador. Así es como se debe pasar un arreglo integrado a una función que *sólo lea* el arreglo integrado, usando la notación de subíndices de arreglos, y *no se modifica* el arreglo integrado. El programa de la figura 8.12 declara la variable apuntador `ptr` de tipo `const int *` (línea 13). Esta declaración se lee de *derecha a izquierda* como “`ptr` es un *apuntador constante a una constante entera*”. La figura muestra los mensajes de error del compilador LLVM de Xcode que se generan cuando se hace un intento de modificar los datos a los que `ptr` apunta (línea 17) y cuando se hace un intento de modificar la dirección almacenada en la variable apuntador (línea 18); esto aparece en las líneas de código con los errores en el editor de texto de Xcode. En la línea 15 no ocurren errores cuando el programa intenta desreferenciar a `ptr`, o cuando el programa trata de imprimir el valor al que `ptr` apunta, ya que *ni* el apuntador *ni* los datos a los que apunta se están modificando en la instrucción.

```

1 // Fig. 8.12: fig08_12.cpp
2 // Intento de modificar un apuntador constante a datos constantes.
3 #include <iostream>

```

Fig. 8.12 | Intento de modificar un apuntador constante a datos constantes (parte 1 de 2).

```

4 using namespace std;
5
6 int main()
7 {
8     int x = 5, y;
9
10    // ptr es un apuntador constante a un entero constante.
11    // ptr siempre apunta a la misma ubicación; el entero
12    // en esa ubicación no se puede modificar.
13    const int *const ptr = &x;
14
15    cout << *ptr << endl;
16
17    *ptr = 7; // error: *ptr es const; no se puede asignar un nuevo valor
18    ptr = &y; // error: ptr es const; no se puede asignar una nueva dirección
19 } // fin de main

```

Mensaje de error del compilador LLVM de Xcode:

Read-only variable is not assignable
Read-only variable is not assignable

Fig. 8.12 | Intento de modificar un apuntador constante a datos constantes (parte 2 de 2).

8.7 Operador sizeof

El operador unario `sizeof` en *tiempo de compilación* determina el tamaño en bytes de un arreglo integrado o de cualquier otro tipo de datos, variable o constante *durante la compilación de un programa*. Cuando se aplica al *nombre* de un arreglo integrado, como en la figura 8.13 (línea 13), el operador `sizeof` devuelve el *número total de bytes en el arreglo integrado* como un valor de tipo `size_t`. La computadora que utilizamos para compilar este programa almacena variables de tipo `double` en 8 bytes de memoria, y `numeros` se declara de forma que tenga 20 elementos (línea 11), por lo que usa 160 bytes en memoria. Cuando se aplica a un *parámetro apuntador* (línea 22) en una función que *recibe un arreglo integrado como argumento*, el operador `sizeof` devuelve el tamaño del *apuntador* en bytes (4 en el sistema que utilizamos); *no el tamaño del arreglo*.



Error común de programación 8.3

Al utilizar el operador `sizeof` en una función para buscar el tamaño en bytes de un *parámetro tipo arreglo*, se obtiene el tamaño en bytes de un *apuntador*, no el tamaño en bytes del *arreglo integrado*.

```

1 // Fig. 8.13: fig08_13.cpp
2 // Al aplicar el operador sizeof al nombre de un arreglo integrado
3 // se devuelve el número de bytes en el arreglo integrado.
4 #include <iostream>
5 using namespace std;
6

```

Fig. 8.13 | Al aplicar el operador `sizeof` al nombre de un arreglo, se devuelve el número de bytes en el mismo (parte 1 de 2).

```

7  size_t getSize( double * ); // prototipo
8
9  int main()
10 {
11     double numeros[ 20 ]; // 20 valores double: ocupa 160 bytes en nuestro sistema
12
13     cout << "El numero de bytes en el arreglo es " << sizeof( numeros );
14
15     cout << "\nEl numero de bytes devueltos por getSize es "
16         << getSize( numeros ) << endl;
17 } // fin de main
18
19 // devuelve el tamaño de ptr
20 size_t getSize( double *ptr )
21 {
22     return sizeof( ptr );
23 } // fin de la función getSize

```

El numero de bytes en el arreglo es 160
El numero de bytes devueltos por getSize es 4

Fig. 8.13 | Al aplicar el operador sizeof al nombre de un arreglo, se devuelve el número de bytes en el mismo (parte 2 de 2).

El número de *elementos* en un arreglo integrado se puede determinar mediante el uso de los resultados de dos operaciones sizeof. Por ejemplo, para determinar el número de elementos en el arreglo integrado numeros, use la siguiente expresión (que se evalúa en *tiempo de compilación*):

```
sizeof numeros / sizeof( numeros[ 0 ] )
```

La expresión divide el número de bytes en numeros (160, suponiendo valores double de 8 bytes) entre el número de bytes en el elemento cero del arreglo integrado (8); el resultado es el número de elementos en numeros (20).

Cómo determinar el tamaño de los tipos fundamentales, un arreglo integrado y un apuntador

En la figura 8.14 se utiliza sizeof para calcular el número de bytes utilizados para almacenar la mayoría de los tipos de datos estándar. La salida se produjo usando la configuración predeterminada en Visual C++ 2012, en una computadora con Windows 7. Los tamaños de los tipos son *dependientes de la plataforma*. Por ejemplo, en otro sistema, double y long double pueden ser de distintos tamaños.

```

1  // Fig. 8.14: fig08_14.cpp
2  // Uso del operador sizeof para determinar los tamaños de los tipos de datos
   estándar.
3  #include <iostream>
4  using namespace std;
5
6  int main()
7  {
8     char c; // variable de tipo char
9     short s; // variable de tipo short

```

Fig. 8.14 | Uso del operador sizeof para determinar tamaños estándar de los tipos de datos (parte 1 de 2).

```

10  int i; // variable de tipo int
11  long l; // variable de tipo long
12  long ll; // variable de tipo long long
13  float f; // variable de tipo float
14  double d; // variable de tipo double
15  long double ld; // variable de tipo long double
16  int array[ 20 ]; // arreglo integrado de int
17  int *ptr = array; // variable de tipo int *
18
19  cout << "sizeof c = " << sizeof c
20      << "\tsizeof(char) = " << sizeof( char )
21      << "\tsizeof s = " << sizeof s
22      << "\tsizeof(short) = " << sizeof( short )
23      << "\tsizeof i = " << sizeof i
24      << "\tsizeof(int) = " << sizeof( int )
25      << "\tsizeof l = " << sizeof l
26      << "\tsizeof(long) = " << sizeof( long )
27      << "\tsizeof ll = " << sizeof ll
28      << "\tsizeof(long long) = " << sizeof( long long )
29      << "\tsizeof f = " << sizeof f
30      << "\tsizeof(float) = " << sizeof( float )
31      << "\tsizeof d = " << sizeof d
32      << "\tsizeof(double) = " << sizeof( double )
33      << "\tsizeof ld = " << sizeof ld
34      << "\tsizeof(long double) = " << sizeof( long double )
35      << "\tsizeof arreglo = " << sizeof arreglo
36      << "\tsizeof ptr = " << sizeof ptr << endl;
37  } // fin de main

```

```

sizeof c = 1      sizeof(char) = 1
sizeof s = 2      sizeof(short) = 2
sizeof i = 4      sizeof(int) = 4
sizeof l = 4      sizeof(long) = 4
sizeof ll = 8     sizeof(long long) = 8
sizeof f = 4      sizeof(float) = 4
sizeof d = 8      sizeof(double) = 8
sizeof ld = 8     sizeof(long double) = 8
sizeof arreglo = 80
sizeof ptr = 4

```

Fig. 8.14 | Uso del operador `sizeof` para determinar tamaños estándar de los tipos de datos (parte 2 de 2).



Tip de portabilidad 8.1

El número de bytes utilizados para almacenar un tipo de datos específico puede variar de un sistema a otro. Al escribir programas que dependan de los tamaños de los tipos de datos, use siempre `sizeof` para determinar el número de bytes utilizados para almacenar los tipos de datos.

El operador `sizeof` se puede aplicar a cualquier expresión o nombre de tipo. Cuando se aplica `sizeof` al nombre de una variable (que no sea el nombre de un arreglo integrado) u otra expresión, se devuelve el número de bytes utilizados para almacenar el tipo específico del valor de la expresión. Los paréntesis utilizados con `sizeof` *sólo* son requeridos si se suministra el nombre de un tipo (por ejemplo, `int`) como su operando. Los paréntesis utilizados con `sizeof` *no* son requeridos cuando el operando de `sizeof` es una expresión. Recuerde que `sizeof` es un operador *en tiempo de compilación*, por lo que no se evalúa su operando.

8.8 Expresiones y aritmética de apuntadores

En esta sección se describen los operadores que pueden tener *apuntadores* como operandos, y cómo se utilizan estos operadores con los apuntadores. C++ permite la **aritmética de apuntadores**: es posible realizar unas cuantas operaciones aritméticas con apuntadores. *La aritmética de apuntadores es apropiada sólo para apuntadores que apuntan a los elementos de arreglos integrados.*

Un apuntador se puede incrementar ($++$) o decrementar ($--$), se puede sumar un entero a un apuntador ($+ \text{o } +=$), se puede restar un entero de un apuntador ($- \text{o } -=$), o se puede restar un apuntador de otro apuntador del mismo tipo; esta operación en particular es apropiada sólo para dos apuntadores que apuntan a elementos del *mismo* arreglo integrado.



Tip de portabilidad 8.2

La mayoría de las computadoras de la actualidad tienen enteros de cuatro o de ocho bytes. Debido a que los resultados de la aritmética de apuntadores dependen del tamaño de los objetos a los que apunta un apuntador, la aritmética de apuntadores es dependiente del equipo.

Suponga que se ha declarado el arreglo `int v[5]` y que su primer elemento se encuentra en la ubicación de memoria 3000. Suponga que se ha inicializado el apuntador `vPtr` para que apunte a `v[0]` (es decir, el valor de `vPtr` es 3000). En la figura 8.15 se muestra un diagrama de esta situación para un equipo con enteros de cuatro bytes. Observe que `vPtr` se puede inicializar para que apunte al arreglo `v` con cualquiera de las siguientes instrucciones (debido a que el nombre de un arreglo integrado es equivalente a la dirección de su elemento cero):

```
int *vPtr = v;
int *vPtr = &v[0];
```



Fig. 8.15 El arreglo integrado `v` y una variable apuntador `int *vPtr` que apunta a `v`.

Sumar enteros a, y restar enteros de apuntadores

En la aritmética convencional, la suma $3000 + 2$ produce el valor 3002. Por lo general éste *no* es el caso con la aritmética de apuntadores. Cuando se suma (o se resta) un entero a un apuntador, éste *no* simplemente se incrementa o decrementa debido a ese entero, sino por ese entero *multiplicado por el tamaño del objeto al que el apuntador hace referencia*. El número de bytes depende del tipo de datos del objeto. Por ejemplo, la instrucción

```
vPtr += 2;
```

produciría 3008 (del cálculo $3000 + 2 * 4$), suponiendo que un `int` se almacena en cuatro bytes de memoria. En el arreglo integrado `v`, `vPtr` apuntaría ahora a `v[2]` (figura 8.16). Si se almacena un entero en ocho bytes de memoria, entonces el cálculo anterior produciría la ubicación de memoria 3016 ($3000 + 2 * 8$).

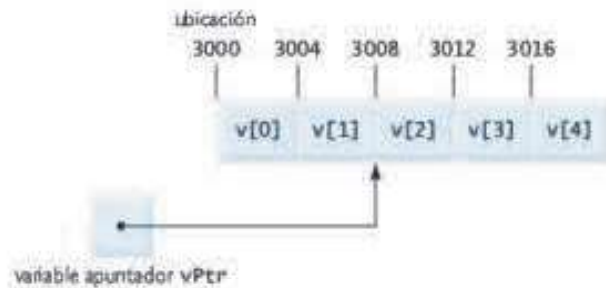


Fig. 8.16 | El apuntador `vPtr` después de la aritmética de apuntadores.

Si `vPtr` se hubiera incrementado a 3016, lo cual apunta a `v[4]`, la instrucción

```
vPtr -= 4;
```

establecería a `vPtr` de vuelta en 3000; el inicio del arreglo integrado. Si un apuntador se va a incrementar o decrementar por uno, se pueden utilizar los operadores de incremento (`++`) y decremento (`--`). Cada una de las instrucciones

```
++vPtr;
vPtr++;
```

incrementa el apuntador para que apunte al *siguiente* elemento del arreglo integrado. Cada una de las instrucciones

```
--vPtr;
vPtr--;
```

decrementa el apuntador para que apunte al elemento *anterior* del arreglo integrado.



Tip para prevenir errores 8.5

En la aritmética de apuntadores no hay comprobación de límites. El programador debe asegurarse de que cada operación aritmética con apuntadores que sume o reste un entero a un apuntador dé como resultado otro apuntador que haga referencia a un elemento dentro de los límites del arreglo integrado.

Resta de apuntadores

Las variables apuntador que apuntan al *mismo* arreglo integrado se pueden restar entre sí. Por ejemplo, si `vPtr` contiene la dirección 3000 y `v2Ptr` contiene la dirección 3008, la instrucción

```
x = v2Ptr - vPtr;
```

asignaría a `x` el número de elementos del arreglo integrado de `vPtr` a `v2Ptr`; en este caso, 2. La aritmética de apuntadores no tiene significado, a menos que se realice en un apuntador que apunte a un arreglo integrado. No podemos asumir que dos variables del mismo tipo se almacenan en forma contigua en la memoria, a menos que sean elementos adyacentes de un arreglo integrado.



Error común de programación 8.4

Restar o comparar dos apuntadores que no hagan referencia a los elementos del mismo arreglo integrado es un error lógico.

Asignación de apuntadores

Un apuntador se puede asignar a otro, si ambos apuntadores son del *mismo* tipo. En caso contrario, se debe usar un operador de conversión de tipos (por lo general, `reinterpret_cast`; lo veremos en la sección 14.7) para convertir el valor del apuntador, que está a la derecha de la asignación, al tipo del apuntador que está a la izquierda de la asignación. La excepción a esta regla es el **apuntador a void** (es decir, `void *`), el cual es un apuntador genérico capaz de representar *cualquier* tipo de apuntador. *Cualquier apuntador a un tipo fundamental o tipo de clase puede asignarse a un apuntador de tipo void * sin necesidad de conversión de tipos.* Sin embargo, un apuntador de tipo `void *` *no se puede* asignar directamente a un apuntador de otro tipo; el apuntador de tipo `void *` debe convertirse primero en el tipo apropiado de apuntador.

No se puede desreferenciar un void *

No se puede desreferenciar un apuntador `void *`. Por ejemplo, el compilador “sabe” que un apuntador a `int` hace referencia a cuatro bytes de memoria en un equipo con enteros de cuatro bytes, pero un apuntador a `void` simplemente contiene una dirección de memoria para un tipo de datos *desconocido*; el compilador *no* conoce el número preciso de bytes a los que hace referencia el apuntador y el tipo de los datos. El compilador debe conocer el tipo de datos para determinar el número de bytes a desreferenciar para un apuntador específico; para un apuntador a `void`, este número de bytes *no se puede* determinar.



Error común de programación 8.5

Asignar un apuntador de un tipo a un apuntador de otro (que no sea `void *`) sin usar una conversión de tipos (por lo general, `reinterpret_cast`) es un error de compilación.



Error común de programación 8.6

Las operaciones permitidas en apuntadores `void *` son: comparar apuntadores `void *` con otros apuntadores, convertir apuntadores `void *` a otros tipos de apuntadores y asignar direcciones a apuntadores `void *`. Todas las demás operaciones en apuntadores `void *` son errores de compilación.

Comparación de apuntadores

Los apuntadores se pueden comparar mediante el uso de los operadores de igualdad y relacionales. Las comparaciones en las que se utilizan operadores relacionales no tienen significado, a menos que los apuntadores apunten a miembros del *mismo* arreglo integrado. Las comparaciones con apuntadores comparan las *direcciones* almacenadas en los apuntadores. Una comparación de dos apuntadores que apunten al mismo arreglo integrado podría mostrar, por ejemplo, que un apuntador apunta a un elemento de mayor numeración del arreglo integrado que el otro apuntador. Un uso común de la comparación de apuntadores es determinar si un apuntador tiene el valor `nullptr`, 0 o `NULL` (es decir, si el apuntador no apunta a nada).

8.9 Relación entre apuntadores y arreglos integrados

Los arreglos integrados y los apuntadores están estrechamente relacionados en C++ y se pueden utilizar de manera *casi* intercambiable. Los apuntadores se pueden utilizar para realizar cualquier operación en la que se involucren los subíndices de arreglos.

Suponga las siguientes declaraciones:

```
int b[5]; // crea el arreglo int b de 5 elementos; b es un apuntador const
int *bPtr; // crea el apuntador int bPtr, que no es un apuntador const
```

Podemos establecer `bPtr` a la dirección del primer elemento en el arreglo integrado `b` con la instrucción

```
bPtr = b; // asigna la dirección del arreglo integrado b a bPtr
```

Esto es equivalente a asignar la dirección del primer elemento del arreglo, como se muestra a continuación:

```
bPtr = &b[ 0 ]; // también asigna la dirección del arreglo integrado b a bPtr
```

Notación apuntador/desplazamiento

El elemento `b[3]` del arreglo integrado se puede referenciar de manera alternativa con la siguiente expresión de apuntador:

```
*{ bPtr + 3 }
```

El 3 en la expresión anterior es el **desplazamiento** para el apuntador. Cuando el apuntador apunta al inicio de un arreglo integrado, el desplazamiento indica a cuál elemento del arreglo integrado se debe hacer referencia, y el valor del desplazamiento es idéntico al subíndice. La notación anterior se conoce como **notación apuntador/desplazamiento**. Los paréntesis son necesarios, debido a que la precedencia de `*` es mayor que la precedencia de `+`. Sin los paréntesis, la expresión anterior sumaría 3 a una copia del valor de `*bPtr` (es decir, se sumaría 3 a `b[0]`, suponiendo que `bPtr` apunta al inicio del arreglo integrado).

Así como se puede hacer referencia al elemento del arreglo integrado con una expresión de apuntador, la *dirección*

```
&b[ 3 ]
```

se puede escribir con la expresión de apuntador

```
bPtr + 3
```

Notación apuntador/desplazamiento con el nombre del arreglo integrado como el apuntador

El nombre del arreglo integrado se puede tratar como apuntador y se puede utilizar en la aritmética de apuntadores. Por ejemplo, la expresión

```
*( b + 3 )
```

también se refiere al elemento `b[3]` del arreglo. En general, todas las expresiones de arreglos integrados con subíndice se pueden escribir con un apuntador y un desplazamiento. En este caso, la notación apuntador/desplazamiento se utilizó con el nombre del arreglo integrado como un apuntador. La expresión anterior *no* modifica el nombre del arreglo integrado; `b` sigue apuntando al primer elemento en el arreglo integrado.

Notación apuntador/subíndice

Los apuntadores pueden usar subíndices de la misma forma que los arreglos integrados. Por ejemplo, la expresión

```
bPtr[ 3 ]
```

hace referencia al elemento `b[1]` del arreglo; esta expresión utiliza la **notación apuntador/subíndice**.

El nombre de un arreglo integrado no se puede modificar

La expresión

```
b += 3
```

produce un error de compilación, ya que trata de *modificar* el valor del nombre del arreglo integrado con la aritmética de apuntadores.



Buena práctica de programación 8.3

Por cuestión de claridad, usamos la notación de arreglos integrados en vez de la notación de apuntadores al manipular arreglos integrados.

Demostración de la relación entre apuntadores y arreglos integrados

En la figura 8.17 se utilizan las cuatro notaciones descritas en esta sección para hacer referencia a los elementos de un arreglo integrado (*notación de subíndice de arreglo*, *notación apuntador/desplazamiento con el nombre del arreglo integrado como apuntador*, *notación de subíndice de apuntador* y *notación apuntador/desplazamiento con un apuntador*) para realizar la misma tarea, a saber, imprimir los cuatro elementos del arreglo integrado de valores `int`, llamado `b`.

```

1 // Fig. 8.17: fig08_17.cpp
2 // Uso de notaciones de subíndice y apuntador con arreglos integrados.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     int b[] = { 10, 20, 30, 40 }; // crea el arreglo integrado b de 4 elementos
9     int *bPtr = b; // establece bPtr para que apunte al arreglo integrado b
10
11     // imprime el arreglo integrado b usando la notación de subíndice de arreglo
12     cout << "Se imprime el arreglo b con:\n\nNotación de subíndice de arreglo\n";
13
14     for ( size_t i = 0; i < 4; ++i )
15         cout << "b[" << i << "] = " << b[i] << "\n";
16
17     // imprime el arreglo integrado b usando el nombre del arreglo y la notación
18     // apuntador/desplazamiento
19     cout << "\nNotación apuntador/desplazamiento en donde "
20         << "el apuntador es el nombre del arreglo\n";
21
22     for ( size_t desplazamiento1 = 0; desplazamiento1 < 4; ++desplazamiento1 )
23         cout << "*(b + " << desplazamiento1 << ") = " << *( b + desplazamiento1 )
24         << "\n";
25
26     // imprime el arreglo integrado b usando bPtr y la notación de subíndice de
27     // arreglo
28     cout << "\nNotación de subíndice de apuntador\n";
29
30     for ( size_t j = 0; j < 4; ++j )
31         cout << "bPtr[" << j << "] = " << bPtr[j] << "\n";
32
33     cout << "\nNotación apuntador/desplazamiento\n";
34
35     // imprime el arreglo integrado b usando bPtr y la notación apuntador/
36     // desplazamiento
37     for ( size_t desplazamiento2 = 0; desplazamiento2 < 4; ++desplazamiento2 )
38         cout << "*(bPtr + " << desplazamiento2 << ") = "
39         << *( bPtr + desplazamiento2 ) << "\n";
40 } // fin de main

```

Se imprime el arreglo b con:

Notación de subíndice de arreglo

b[0] = 10

b[1] = 20

b[2] = 30

b[3] = 40

Fig. 8.17 | Uso de notaciones de subíndice y apuntador con los arreglos integrados (parte 1 de 2).

```

Notación apuntador/desplazamiento en donde el apuntador es el nombre del arreglo
*(b + 0) = 10
*(b + 1) = 20
*(b + 2) = 30
*(b + 3) = 40

Notación de subíndice de apuntador
bPtr[0] = 10
bPtr[1] = 20
bPtr[2] = 30
bPtr[3] = 40

Notación apuntador/desplazamiento
*(bPtr + 0) = 10
*(bPtr + 1) = 20
*(bPtr + 2) = 30
*(bPtr + 3) = 40

```

Fig. 8.17 | Uso de notaciones de subíndice y apuntador con los arreglos integrados (parte 2 de 2).

8.10 Cadenas basadas en apuntador

Ya utilizamos la clase `string` de la Biblioteca estándar de C++ para representar las cadenas como objetos completos. Por ejemplo, el caso de estudio de la clase `LibroCalificaciones` en los capítulos 3 a 7 representa el nombre de un curso que usa un objeto `string`. En el capítulo 21 (en inglés en el sitio web) se presenta la clase `string` con detalle. En esta sección introduciremos las cadenas basadas en apuntador estilo C (como se definen mediante el lenguaje de programación C), a las que simplemente llamaremos **cadenas de C**. La clase `string` de C++ es preferida para usarse en nuevos programas, ya que elimina muchos de los problemas de seguridad y errores que pueden ocurrir al manipular cadenas de C. Aquí cubriremos las cadenas de C para una comprensión más detallada de los apuntadores y los arreglos integrados. Además, si trabaja con programas heredados de C y C++, es probable que se encuentre con cadenas basadas en apuntador. En el apéndice F cubriremos con detalle las cadenas de C.

Caracteres y constantes tipo carácter

Los caracteres son los bloques de construcción fundamentales de los programas de código fuente de C++. Todo programa está compuesto de una secuencia de caracteres que (al agruparse de una manera significativa) el compilador interpreta como una serie de instrucciones que se utilizan para realizar una tarea. Un programa puede contener **constantes tipo carácter**. Una constante carácter es un valor entero que se representa como un carácter entre comillas sencillas. El *valor* de una constante carácter es el valor entero del carácter en el conjunto de caracteres del equipo. Por ejemplo, 'z' representa el valor entero de z (122 en el conjunto de caracteres ASCII; vea el apéndice B), y '\n' representa el valor entero de nueva línea (10 en el conjunto de caracteres ASCII).

Cadenas de caracteres

Una cadena es una serie de caracteres que se trata como una sola unidad. Una cadena puede incluir letras, dígitos y diversos **caracteres especiales** tales como +, -, *, / y \$. Las **literales de cadena**, o **constantes de cadena** en C++ se escriben entre dobles comillas, como se muestra a continuación:

"John Q. Doe"	(un nombre)
"9999 Main Street"	(una calle)
"Maynard, Massachusetts"	(una ciudad y un estado)
"(201) 555-1212"	(un número telefónico)